NLP Environment Setup & Duplicate Question Detector Guide (Revised)

Complete Setup, Troubleshooting Log & Project Roadmap (From Start to Verification)

1. Overview & Workspace Architecture

To maintain a clean and scalable development environment, all Natural Language Processing (NLP) projects are consolidated within a single root directory named `nlp_projects`. Rather than duplicating massive machine learning dependencies (such as PyTorch, Transformers, and spaCy) across multiple project folders—which quickly consumes tens of gigabytes of disk space—a single persistent virtual environment (`nlp-env`) serves the entire ecosystem.

This architecture ensures isolated dependency management while maximizing storage efficiency across foundational NLP workflows including vector search, text summarization, and named entity recognition.

Directory Structure

t
C:\Users<YourUsername>\nlp_projects
├── nlp-env\ # Isolated Python 3.12 virtual environment
├── duplicate_question_detector\ # Project 1: Vector Search & Semantic Similarity
│ └── app.py
├── content_summarization\ # Project 2: Abstractive vs. Extractive Summarization
└── resume_parsing\ # Project 3: NER & Information Extraction

```
None

# 2. Lessons Learned & Troubleshooting Log (Why Python 3.12?)

Building a stable modern NLP environment on Windows requires navigating native
C-extension compilation and wheel availability. Below is the technical breakdown
of initialization challenges and verified solutions.

### 1. Python 3.15 Compatibility Issue
```

```
* **Issue:** Python 3.15.2 is too novel for many key machine learning packages. Pre-built binary wheels (`.whl`) are not yet published on PyPI for fundamental dependencies such as `PyYAML` and various C-extensions.
* **Failure Mode:** Installing via `pip` forces fallback to source distribution builds (`.tar.gz`), which crash with the compiler error: `Microsoft Visual C++ 14.0 or greater is required`.

### 2. Solution – Switching to Python 3.12
* **Resolution:** Installed Python 3.12 alongside existing versions using the Python Install Manager (`py install 3.12`).
* **Environment Provisioning:** Rebuilt the target environment explicitly with `py -3.12 -m venv nlp-env`. Python 3.12 has complete pre-built binary wheel coverage across PyTorch, Hugging Face, and spaCy, eliminating compiler build steps entirely.

### 3. Network Timeout Resilience
* **Issue:** Large distribution packages like `torch` (~2GB) and heavyweight scientific libraries frequently hit socket read timeouts over standard connections when using default `pip` configurations.
* **Resolution:** Executed package installations with explicit network parameters: `--default-timeout=1000 --retries 10 --prefer-binary`.

### 4. Windows Symlink Warning
* **Observation:** Hugging Face Hub logs a warning on Windows systems indicating that file symlinking is unavailable without Developer Mode enabled.
* **Impact:** Non-critical. Hugging Face automatically falls back to standard file copying in cache directories. Model loading and execution proceed without functional impairment.

# 3. Step-by-Step Environment Setup (Verified Working Workflow)

Follow these steps sequentially in PowerShell to provision the unified workspace.

### Step 1: Install & Verify Python 3.12
Install Python 3.12 via the launcher and confirm installed runtimes:

```powershell
py install 3.12
py --list
```
```

Expected Output: The listing shows both 3.15 and 3.12 installed on the system.

Step 2: Create the Workspace & Virtual Environment

Navigate to your desired parent folder, construct the workspace, and initialize the Python 3.12 environment:

```
None
mkdir nlp_projects
cd nlp_projects
py -3.12 -m venv nlp-env
```

Step 3: Activate the Virtual Environment

Activate the environment within PowerShell:

```
None
.\nlp-env\Scripts\Activate.ps1
```

Verification: The prompt prefix updates to `(nlp-env)`.

Note: If PowerShell prevents script execution, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` in an elevated shell.

Step 4: Upgrade Packaging Tools & Install Core Libraries

Upgrade baseline tooling and install foundational transformer packages:

```
None
python -m pip install --upgrade pip setuptools wheel
pip install --default-timeout=1000 --retries 10 --prefer-binary transformers
sentence-transformers
```

Step 5: Optional Libraries for Upcoming Projects & Deployment

Install dependencies required for summarization, entity extraction, web APIs, and database integration:

```
None
pip install --default-timeout=1000 --retries 10 --prefer-binary spacy sumy fastapi
uvicorn streamlit psycopg2-binary
```

```
python -m spacy download en_core_web_sm
```

4. Environment Verification (`app.py`)

To confirm that PyTorch, Transformers, and Hugging Face network connections function correctly end-to-end, create an initial script at `duplicate_question_detector/app.py`.

Verification Script

Python
```
from transformers import pipeline

print("Loading duplicate question detection pipeline...")
classifier = pipeline("feature-extraction",
model="sentence-transformers/all-MiniLM-L6-v2")
print("Environment verified successfully! Transformers and PyTorch are ready to
build.")
```

Execution & Expected Output

Run the script from your active environment terminal:I
python duplicate_question_detector/app.py

None

Expected Console Output:

```text
Checking PyTorch integration...
PyTorch Tensor test: tensor([1., 2., 3.])
Checking Hugging Face Transformers...
No model was supplied, defaulting to
distilbert-base-uncased-finetuned-sst-2-english...
Pipeline Result: [{'label': 'POSITIVE', 'score': 0.9998}]

Environment verified successfully! Transformers and PyTorch are ready to build.
```

5. Project 1 Implementation Blueprint — Duplicate Question Detection

Objective: Develop a semantic duplicate detector leveraging vector representations and cosine similarity via sentence-transformers (`all-MiniLM-L6-v2`), laying the foundation for scalable vector database search.

Step 1: Load Model & Reference Questions

```python
from sentence_transformers import SentenceTransformer, util

# Load lightweight pre-trained sentence transformer model
model = SentenceTransformer('all-MiniLM-L6-v2')

# In-memory database of existing reference questions
existing_questions = {
    1: "How to sort a list in Python?",
    2: "What's the difference between list.sort() and sorted()?",
    3: "How to concatenate strings in Python?",
    4: "Merge multiple lists in Python"
}
```

Step 2: Encode Reference Questions Offline

Generate dense 384-dimensional vector embeddings for all stored questions:n
question_embeddings = {
q_id: model.encode(text) for q_id, text in existing_questions.items()
}

```
None

### Step 3: Build Cosine Similarity Matching Function

Compare incoming user queries against stored vector representations:

```python
def find_duplicate(new_question, threshold=0.75):
    new_embedding = model.encode(new_question)
    candidates = []
```
```

```
    for q_id, existing_embedding in question_embeddings.items():
        similarity = util.pytorch_cos_sim(new_embedding,
existing_embedding).item()
        if similarity > threshold:
            candidates.append({
                "question_id": q_id,
                "question_text": existing_questions[q_id],
                "similarity": round(similarity, 4)
            })

    candidates.sort(key=lambda x: x["similarity"], reverse=True)
    return candidates
```

Step 4: Test & Scale to Production

1. **Local Evaluation:** Test queries against the function:

n
test_query = "How do I sort a list alphabetically in Python?"
matches = find_duplicate(test_query, threshold=0.70)
print(f"Matches for query: '{test_query}'")
for match in matches:
print(f" - [ID: {match['question_id']}] {match['question_text']} (Score: {match['similarity']})")

```
None

2. **Production Scaling Strategy:**
   * **Database Integration:** Store dense vectors in PostgreSQL utilizing the
`pgvector` extension to handle millions of records efficiently.
   * **API Layer:** Wrap the search function inside a high-performance `FastAPI`
endpoint to serve inference requests to front-end clients or microservices.

# 6. Upcoming NLP Projects Roadmap

| Project | Focus Area | Key Technologies & Models | Architecture / Deliverables |
|---|---|---|---|
| **Project 2: Content Summarizer** | Abstractive vs. Extractive Summarization |
`facebook/bart-large-cnn`, `sumy` (TextRank), Streamlit | Interactive dashboard
allowing users to compare deep learning abstractive summaries against statistical
extractive text rankings. |
```

```
| **Project 3: Resume Parser & Matcher** | Named Entity Recognition (NER) & Information Extraction | spaCy (`en_core_web_sm`), Regex, Python Set Operations | Structured extraction of candidate contact info, educational history, and skill matrices matched against job description criteria. |
```